Reasoning and explaining all year round

A mathematical statement has a domain and hypotheses. Before proving it, write them down; to refute it, give an example that satisfies the hypotheses and contradicts the conclusion. A figure or a simulation helps the search but does not prove a universal statement.

Tool	Task and output	Targeted review
Membership and sets	Describe a line as the set of points; check all coordinates.	Application 1, p. 90.
Quantifiers and negation	Negate a universal positivity statement and give a witness index.	R10; application 2.
Implication and conv.	Distinguish fixed point and convergence; give a divergent sequence.	Application 4.
Equivalence	Solve a logarithmic equation after setting the domain.	Application 1.
Proof by induction	Identify the property, initialise and prove the inductive step at an arbitrary index.	Applications 3 then 5.
Counterexample	Remove one hypothesis of the IVT; show what fails.	Application 5.
Contrapositive	Show that two vectors whose dot product is non-zero are not orthogonal.	R60 and R61.
Contradiction	Suppose the four vertices of a non-flat tetrahedron lie in one plane.	Unit 10.2.
Case analysis	Deal with a zero parameter before dividing.	Application 5.
Sufficient condition	Distinguish a size guaranteed by a bound on an exact minimum size.	Applications 4 and 5.

A reusable oral protocol

In one minute: state the object studied and the conclusion, name the tool and its assumptions, give the decisive step, then a check. The partner listens without interrupting and reformulates the conclusion ; a question then asks why an assumption is needed. The "One-minute talk" tasks in each chapter serve as material.

Brief feedback from the listener

Is the object defined? Is the key assumption named? Is the link between the steps understandable? Is the result interpreted? Choose a single point to improve before start again.

Results assumed and proofs to reuse

Operations on and composition of limits, monotone convergence, the IVT, the general equivalent characterisations of convexity, the general existence of antiderivatives and the additivity of variance under independence are assumed within the framework given in class. The proofs listed are given in full and are the subject of reproduction or adaptation. A proof of the IVT by bisection, the method of Newton, oblique asymptotes and combinations with repetition are possible extension topics: knowing them is not a hidden prerequisite of the proposed assessments.

Practical-work extension: compare the samples

The program below extends P5-P6. The tolerance in the counting is only there to absorb the decimal representation of floats; the exact calculation uses rational fractions.

```
"""Compare the means of several independent samples."""
from random import Random
from statistics import mean, pstdev
from math import sqrt, comb
from fractions import Fraction

def experience(n, repetitions=1000, p=0.5, graine=2026):
    if n <= 0 or repetitions < 2 or not 0 <= p <= 1:
        raise ValueError("Parameters outside the domain")
    rng = Random(graine)
    moyennes = [sum(rng.random() < p for _ in range(n))/n
                for _ in range(repetitions)]
    return {"n": n, "moyenne": mean(moyennes),
            "ecart_type_observe": pstdev(moyennes),
            "ecart_type_theorique": sqrt(p*(1-p)/n),
            "proportion_ecart_au_moins_0_1":
                sum(abs(m-p) >= 0.1-1e-12 for m in moyennes)/repetitions}

def ecart_exact_equilibre(n, delta):
    """delta is a Fraction: integer boundaries are exact."""
    if n <= 0 or delta <= 0:
        raise ValueError("n and delta must be positive")
    favorables = sum(comb(n,k) for k in range(n+1)
                     if abs(Fraction(k,n)-Fraction(1,2)) >= delta)
    return Fraction(favorables,2**n)

if __name__ == "__main__":
    import json
    print(json.dumps([experience(n) for n in (10,100,1000)],indent=2))
```

Algorithms · P1 to P6

These programs are examples from the notes. To understand them, state the inputs, the output and the assumptions, then carry out a few steps by hand. The answers to the activities and homework remain to be written up separately.

P1 · Terms, sum and threshold

Suppose n is a non-negative integer and the safety cap a positive integer. The first function returns u_n and $\sum_{k=0}^{n} u_k$ for $u_{k+1} = au_k + b$. The second seeks a first upward crossing; its safety cap only signals that it was not found within the set number of steps.

```
def terme_somme(n, u0, a, b):
    u, total = u0, u0
    for k in range(n):
        u = a*u + b
        total += u
    return u, total

def seuil_croissant(u0, a, b, cible, maximum=100000):
    u, n = u0, 0
    while u <= cible and n < maximum:
        u = a*u + b
        n += 1
    if u <= cible:
        return None
    return n
```

P2 · Bisection

The calculations below use floating-point numbers; the mathematical guarantee assumes the signs correctly determined. Assumptions: f continuous on $[a, b]$ and $f(a)f(b) \leq 0$. To identify a unique zero, prove uniqueness separately. The function returns an interval of length at most ε, as long as machine precision still allows the interval to be split.

```
def dichotomie(f, a, b, epsilon):
    assert a < b and epsilon > 0
    fa, fb = f(a), f(b)
    assert fa*fb <= 0
    if fa == 0: return a, a
    if fb == 0: return b, b
    while b-a > epsilon:
        m = (a+b)/2
        if m == a or m == b:
            raise ArithmeticError("machine precision reached")
        fm = f(m)
        if fm == 0: return m, m
        if fa*fm < 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return a, b
```

P3 · Euler

With N a strictly positive integer and $t_0 < \text{fin}$, for $y' = F(t,y)$, $Y_{k+1} = Y_k + hF(t_k, Y_k)$. The program returns all approximations; it does not prove their accuracy.

```
def euler(F, t0, y0, fin, N):
    assert N > 0 and fin > t0
    h = (fin-t0)/N
    y, valeurs = y0, [(t0, y0)]
    for k in range (N):
        t = t0+k*h
        y = y+h*F(t, y)
        valeurs.append((t0+(k+1)*h, y))
    return valeurs
```

P4 · Rectangles

For N a strictly positive integer and f continuous and increasing on $[a,b]$, the result gives a lower bound and an upper bound for the integral. Without monotonicity, these are only two approximations to examine.

```
def rectangles(f, a, b, N):
    assert N > 0 and a < b
    h = (b-a)/N
    gauche = h* sum(f(a+k*h) for k in range (N))
    droite = h* sum(f(a+k*h) for k in range (1, N+1))
    return gauche, droite
```

P5 · Binomial distribution and simulation

Suppose n is a non-negative integer, $0 \leq p \leq 1$ and k is an integer. The direct calculation below is suitable for the modest sizes used in the exercises; for very large parameters, specialised numerical libraries are preferable.

```
from math import comb
from random import Random

def probabilite(n, p, k):
    if k < 0 or k > n: return 0.0
    return comb(n, k)*p**k*(1-p)**(n-k)

def cumul(n, p, k):
    return sum(probabilite(n, p, j)
               for j in range(min(k, n)+1))

def simuler_binomiale(n, p, generateur):
    return sum(generateur.random() < p for _ in range(n))

rng = Random(2026)
```

P6 · Fluctuations of a frequency

Suppose N is a strictly positive integer and $0 \leq p \leq 1$. The seed reproduces the same example. Repeat with other seeds; the law of large numbers does not say that the observed deviation decreases at each step.

```
from random import Random

def frequences(N, p, graine=2026):
    rng = Random(graine)
    succes, resultat = 0, []
    for n in range(1, N+1):
        succes += rng.random() < p
        resultat.append(succes/n)
    return resultat
```

Catalogue of proofs

This list shows where to find the proofs in the notes. It does not fix a universal number of " required proofs": the groupings vary depending on the presentation.

Proved result	Proof
Comparing two sequences at infinity	p. 16
Increasing and not bounded above	p. 16
Bernoulli and geometric limits	p. 16
Limits of the exponential	p. 23
Powers and the exponential	p. 23
Convexity and tangents	p. 37
Limit of x ln(x) at zero	p. 52
Antiderivative given by an area, increasing case	p. 73
Integration by parts	p. 73
Pascal by counting	p. 30
Pascal by calculation	p. 30
Number of subsets: double counting	p. 30
Minimum distance to the plane	p. 59
Cartesian equation of a plane	p. 89
Probability of k successes	p. 44
Binomial expectation and variance	p. 81
Chebyshev on a finite sample space	p. 81
Derivative of the logarithm (differentiability assumed)	p. 52
Two antiderivatives differ by a constant	p. 67
Solutions of $y' = ay$	p. 68
Concentration of the mean, then the law of large numbers	p. 81

End-of-year and oral workshops

O1 · Turning a topic into a question

S82 · 2 h. Choose a situation that allows reasoning from the syllabus, then write a specific question. Examples: "How do we choose a number of trials to make failure rare?"; " How does a cooling model predict a waiting time?"; "Why does a mean become more stable as the sample grows?" Produce a page containing the question, the assumptions, a numerical example, an accessible proof and a limitation of the model. Allow 20 min of individual research, 30 min of mathematical work, 30 min of discussion in groups, 25 min of writing and 15 min of feedback.

O2 · Presenting and answering

S84 · 2 h. Prepare a 10-minute presentation and answers to the following questions: why this model ? which assumption is decisive? what does each variable mean? which result is proved? what changes for another value of the parameter? Practise in groups of three: a speaker, a listener and an observer. The observer notes one clear step, one step needing justification and one wording to improve. Swap roles. A video is only useful if it is permitted and actually watched again.

O3 · Mock oral exam

S87 and S93. Prepare two questions and give a presentation followed by questioning. Use a simple grid: mathematical accuracy, structure, explanation of notation, response to objections, quality of the oral exchange. The final arrangements are those given in the instructions and notices for the exam session.